PARTNER TRUCK VISIBILITY & LOGISTICS COORDINATION PLATFORM

Complete System Architecture Blueprint

Technical design, infrastructure layers, security architecture, and scalability strategy for enterprise logistics orchestration.

Document Version	Final v1.0
Classification	Internal Technical Document
Date	June 2026
Status	Approved for Development

1. Executive Overview

This document provides the complete technical and operational architecture of a logistics coordination platform designed for manufacturers, logistics managers, dispatchers, trucking partners, and drivers. The platform centralizes truck visibility, dispatch operations, partner coordination, GPS tracking, operational analytics, real-time communication, and multi-tenant SaaS infrastructure.

The architecture is designed to scale from a single manufacturing company into a regional logistics network, supporting:

- **Real-time truck visibility** across partner fleets
- **Automated dispatch orchestration** with intelligent matching
- **Live GPS tracking** with geofencing and route optimization
- **Operational analytics** for fleet utilization and performance
- **Multi-tenant SaaS** with enterprise-grade security
- **Mobile-first driver experience** with offline capability

Strategic Positioning: The platform does not compete with consumer delivery apps (Lalamove, Transportify) but instead focuses on enterprise fleet visibility, partner coordination, internal logistics orchestration, and operational intelligence. This creates strong B2B SaaS opportunities with recurring revenue, high switching costs, network effects, and regional scalability.

2. High-Level System Architecture

The platform consists of six major architectural layers, each designed for horizontal scalability and fault tolerance:

2.1. Frontend Layer

Web dashboards, mobile driver app, partner portal, admin panel — built with Next.js and React Native for cross-platform consistency.

2.2. API Layer

RESTful and GraphQL APIs handling authentication, dispatch, tracking, and analytics — secured via JWT and OAuth 2.0.

2.3. Core Business Logic Layer

Dispatch engine, matching engine, status engine, and notification engine — the operational brain of the platform.

2.4. Real-time Infrastructure

WebSockets, GPS streaming pipelines, and event queues — ensuring sub-second latency for critical updates.

2.5. Database Layer

Operational PostgreSQL cluster, analytics data warehouse, time-series tracking logs, and audit trails.

2.6. External Services

Maps (Google/Mapbox), SMS (Twilio), email (SendGrid), push notifications (Firebase), and ERP integrations.

Architecture Principles: Event-driven design, CQRS (Command Query Responsibility Segregation) for read/write separation, eventual consistency for analytics, and strong consistency for transactional operations (dispatch assignments, status updates).

3. User Roles & Permission Matrix

The platform implements Role-Based Access Control (RBAC) with granular permissions across five primary user roles:

Role	Primary Responsibilities	System Access Level
Super Admin	Manage platform, companies, subscriptions, global settings	Full system access
Logistics Manager	Monitor trucks, assign dispatches, view analytics, manage partners	Company-wide operational access
Dispatcher	Coordinate daily deliveries, match trucks to loads, monitor active trips	Operational dispatch access
Partner Admin	Manage trucking fleets, register drivers, update fleet status	Partner fleet management
Driver	Receive jobs, update trip progress, upload proof of delivery, GPS tracking	Mobile app limited access

Table 3.1: User role definitions and access scope matrix

Permission Granularity: Each role supports sub-permissions (e.g., 'view-only analytics' vs. 'export analytics') and can be customized per company tenant. Multi-tenant isolation ensures one company's data is never visible to another.

4. Database Architecture

The data layer uses a polyglot persistence strategy optimized for different access patterns:

4.1 Core Relational Schema

Primary entities and their relationships:

- **companies** — Tenant isolation boundary; each manufacturer or logistics operator
- **partners** — Trucking partner organizations with fleet registration
- **trucks** — Vehicle registry with specifications, GPS coordinates, operational status
- **drivers** — Driver profiles, licenses, ratings, assignment history
- **dispatches** — Delivery assignments linking trucks, drivers, routes, and cargo
- **routes** — Predefined and dynamic route definitions with waypoints
- **deliveries** — Individual delivery legs within a dispatch lifecycle
- **tracking_logs** — Time-series GPS and status event streams
- **maintenance_records** — Scheduled and ad-hoc maintenance tracking
- **notifications** — Multi-channel notification queue and delivery status
- **users** — Platform user accounts with RBAC bindings
- **permissions** — Granular permission definitions and role mappings
- **audit_logs** — Immutable compliance and security audit trail

Key Relationships: One company has many dispatches. One partner owns many trucks. One truck has many trips. One driver handles many dispatches. All relationships enforce referential integrity and cascade rules appropriate to business logic.

4.2 Core Database Tables

The following tables represent the foundational schema:

Table: trucks

Column	Type	Constraints	Description
id	UUID	PK, Auto-gen	Unique vehicle identifier
partner_id	UUID	FK → partners	Owning partner organization
company_id	UUID	FK → companies	Tenant company (multi-tenancy)
plate_number	VARCHAR(20)	Unique, Not Null	Official vehicle registration

truck_type	ENUM	Not Null	Wing van, 10-wheeler, 6-wheeler, etc.
capacity_kg	DECIMAL(10,2)	Not Null	Maximum cargo capacity in kilograms
capacity_cbm	DECIMAL(8,2)	Nullable	Volume capacity in cubic meters
status	ENUM	Default: available	available, loading, in_transit, returning, maintenance, reserved
gps_latitude	DECIMAL(10,8)	Nullable	Last known latitude
gps_longitude	DECIMAL(11,8)	Nullable	Last known longitude
gps_accuracy	DECIMAL(5,2)	Nullable	GPS accuracy in meters
last_updated	TIMESTAMP	Auto-update	Last status/GPS update timestamp
created_at	TIMESTAMP	Default NOW()	Record creation timestamp
is_active	BOOLEAN	Default TRUE	Soft delete flag

Table: dispatches

Column	Type	Constraints	Description
id	UUID	PK, Auto-gen	Unique dispatch identifier
company_id	UUID	FK → companies	Tenant company
truck_id	UUID	FK → trucks, Nullable	Assigned vehicle
driver_id	UUID	FK → drivers, Nullable	Assigned driver
origin	JSONB	Not Null	Origin location with lat/lng/address
destination	JSONB	Not Null	Destination location with lat/lng/address
waypoints	JSONB	Nullable	Intermediate stops array
cargo_type	VARCHAR(100)	Not Null	Cargo classification
cargo_weight	DECIMAL(10,2)	Nullable	Actual cargo weight
status	ENUM	Default: pending	pending, assigned, loading, in_transit, arrived, delivered, cancelle
priority	ENUM	Default: normal	low, normal, high, urgent
assigned_at	TIMESTAMP	Nullable	Assignment timestamp
started_at	TIMESTAMP	Nullable	Trip start timestamp
completed_at	TIMESTAMP	Nullable	Delivery completion timestamp
estimated_duration	INTEGER	Nullable	ETA in minutes
actual_duration	INTEGER	Nullable	Actual trip duration

created_by	UUID	FK → users	Dispatcher who created
created_at	TIMESTAMP	Default NOW()	Record creation
updated_at	TIMESTAMP	Auto-update	Last modification

Table 4.1 & 4.2: Core entity schemas with full column specifications

Indexing Strategy: Composite indexes on (company_id, status) for dashboard queries, spatial indexes on GPS coordinates for geo-queries, and partial indexes on active dispatches for performance.

5. Realtime Operational Flow

The platform's real-time capabilities follow a structured nine-step operational pipeline:

STEP 1: Partner Fleet Update

Partner admin registers trucks and updates fleet status via web portal or bulk CSV import. Each truck entry triggers validation against partner quota and company subscription limits.

STEP 2: Realtime Status Push

Truck status changes are pushed into the real-time event bus (Redis Pub/Sub → Socket.IO). Status transitions are validated by the Status Engine to prevent invalid state changes.

STEP 3: Dashboard Synchronization

Logistics manager dashboard updates instantly via WebSocket listeners. The dashboard renders available trucks, active trips, and fleet utilization metrics with sub-second latency.

STEP 4: Dispatch Assignment

Logistics manager or dispatcher creates a dispatch request specifying origin, destination, cargo type, capacity requirements, and priority.

STEP 5: Dispatch Engine Validation

The Dispatch Engine validates truck availability, driver eligibility, route feasibility, and capacity matching. Conflicts trigger automatic resolution or manual override workflows.

STEP 6: Driver Notification

Assigned driver receives trip details via mobile push notification, SMS fallback, and in-app job card. Driver must acknowledge assignment within configurable timeout (default: 15 minutes).

STEP 7: GPS Tracking Pipeline

Driver mobile app streams GPS coordinates every 30 seconds (configurable) via WebSocket. Tracking Service processes streams, applies geofencing logic, and detects anomalies (deviation, stoppage).

STEP 8: Delivery Lifecycle Events

Standardized event stream: Loading → In Transit → Arrived → Delivered. Each transition requires driver confirmation with optional proof (photo, signature, QR scan).

STEP 9: Analytics Recording

Analytics engine records operational metrics: dispatch time, turnaround time, fuel estimates vs. actual, delay reasons, and partner performance scores.

Event-Driven Guarantees: All status transitions generate immutable audit events. Event sourcing pattern ensures complete operational history reconstruction for compliance and dispute resolution.

6. Realtime Infrastructure

Real-time architecture is critical because logistics operations require live situational awareness. The platform implements a multi-tier real-time strategy:

6.1 Recommended Technology Stack

- **Socket.IO:** Primary WebSocket transport for bidirectional event streaming between server and clients
- **Redis Pub/Sub:** In-memory message broker for horizontal scaling of WebSocket servers
- **Supabase Realtime:** PostgreSQL change data capture for database-driven real-time updates
- **WebSockets (Native):** Fallback and high-frequency GPS streaming channels
- **Apache Kafka:** Event streaming backbone for analytics pipeline and cross-service communication

6.2 Core Realtime Events

Standardized event taxonomy ensures consistent handling across all services:

Event Name	Trigger Condition	Payload Schema
truck.status.updated	Triggered when truck transitions between operational states	JSON Schema v1.0
dispatch.created	New dispatch assignment initiated	JSON Schema v1.0
dispatch.assigned	Truck and driver confirmed for dispatch	JSON Schema v1.0
trip.started	Driver begins transit phase	JSON Schema v1.0
gps.updated	New GPS coordinate batch received	JSON Schema v1.0
delivery.completed	Final proof of delivery confirmed	JSON Schema v1.0
delivery.delayed	ETA exceeded threshold, alert triggered	JSON Schema v1.0
maintenance.due	Preventive maintenance threshold reached	JSON Schema v1.0
partner.fleet.updated	Partner bulk fleet status change	JSON Schema v1.0

Table 6.1: Standardized real-time event taxonomy

7. API Architecture

The platform exposes a comprehensive REST API with versioning, rate limiting, and comprehensive documentation via OpenAPI 3.0 specification.

7.1 Authentication APIs

- **POST /api/v1/auth/login** — Email/password or SSO login; returns JWT access + refresh tokens
- **POST /api/v1/auth/register** — Company onboarding with admin account creation
- **POST /api/v1/auth/refresh** — Token refresh using valid refresh token
- **POST /api/v1/auth/logout** — Token revocation and session cleanup
- **POST /api/v1/auth/forgot-password** — Password reset workflow via email
- **GET /api/v1/auth/me** — Current user profile and permissions

7.2 Truck APIs

- **GET /api/v1/trucks** — List trucks with filtering (status, type, partner, location radius)
- **POST /api/v1/trucks** — Register new truck (Partner Admin only)
- **GET /api/v1/trucks/{id}** — Retrieve truck details with current status and history
- **PATCH /api/v1/trucks/{id}/status** — Update truck operational status
- **PATCH /api/v1/trucks/{id}/gps** — Update GPS coordinates (driver mobile app)
- **DELETE /api/v1/trucks/{id}** — Soft delete truck (Super Admin only)

7.3 Dispatch APIs

- **POST /api/v1/dispatches** — Create new dispatch assignment
- **GET /api/v1/dispatches** — List dispatches with status, date range, partner filters
- **GET /api/v1/dispatches/{id}** — Full dispatch details with timeline and tracking
- **PATCH /api/v1/dispatches/{id}** — Update dispatch (status, driver, truck, priority)
- **POST /api/v1/dispatches/{id}/assign** — Assign truck and driver to dispatch
- **POST /api/v1/dispatches/{id}/cancel** — Cancel dispatch with reason logging
- **GET /api/v1/dispatches/{id}/timeline** — Complete event timeline for audit

7.4 Tracking APIs

- **POST /api/v1/gps/batch** — Batch GPS coordinate upload from mobile app

- **GET /api/v1/trips/{id}/live** — Real-time trip tracking with current position and ETA
- **GET /api/v1/trips/{id}/history** — Historical GPS path reconstruction
- **GET /api/v1/trucks/{id}/nearby** — Find trucks within specified radius of coordinates

7.5 Analytics APIs

- **GET /api/v1/analytics/utilization** — Fleet utilization metrics by time period
- **GET /api/v1/analytics/delays** — Delay analysis with root cause categorization
- **GET /api/v1/analytics/partner-performance** — Partner reliability and turnaround scores
- **GET /api/v1/analytics/fuel-efficiency** — Fuel consumption estimates vs. actuals
- **GET /api/v1/analytics/dashboard** — Consolidated KPI dashboard data

API Standards: All APIs use JSON request/response bodies, HTTPS only, implement pagination (cursor-based for real-time data), and enforce rate limiting (1000 req/min for standard, 5000 req/min for enterprise tiers).

8. Security Architecture

Enterprise-grade security is foundational to the platform, protecting sensitive logistics data and ensuring operational integrity:

8.1 JWT Authentication

Stateless token-based auth with short-lived access tokens (15 min) and long-lived refresh tokens (7 days). RSA-256 signed tokens with key rotation.

8.2 Role-Based Access Control (RBAC)

Granular permissions mapped to roles with resource-level access control. Permission checks enforced at API gateway and service layer.

8.3 Multi-Tenant Isolation

Row-level security in PostgreSQL ensures one company's operational data is never visible to another. Tenant ID validated on every database query.

8.4 HTTPS Encryption

TLS 1.3 for all communications. HSTS headers, perfect forward secrecy, and certificate pinning on mobile apps.

8.5 Audit Logging

Immutable audit trail capturing all data mutations, access events, and permission changes. Stored in append-only table with cryptographic verification.

8.6 API Rate Limiting

Tiered rate limits per tenant and user role. DDoS protection via Cloudflare and application-level throttling.

8.7 Secure GPS Event Validation

GPS coordinates validated against expected route corridors. Anomaly detection flags spoofed or impossible location jumps.

8.8 Backup & Disaster Recovery

Point-in-time recovery (PITR) with 7-day retention. Cross-region replication for critical data. RPO: 5 minutes, RTO: 30 minutes.

8.9 Data Encryption at Rest

AES-256 encryption for database volumes and backup storage. Key management via cloud KMS with rotation policies.

8.10 Input Validation & Sanitization

Strict schema validation, SQL injection prevention via parameterized queries, XSS protection, and CSRF tokens for web clients.

> **Critical Security Consideration:** One company's operational data must never be visible to another company. This is enforced at the database query level (RLS policies), API middleware (tenant validation), and application layer (service method scoping). Any cross-tenant data leakage is treated as a P0 security incident.

9. Scalability Strategy

The platform follows a phased scaling approach aligned with business growth:

Phase 1: Single Company

1-50 trucks, single tenant. Monolithic deployment on managed VPS. PostgreSQL primary with read replica. Redis for caching and sessions.

Phase 2: Multiple Partners

50-500 trucks, 5-20 partners. Containerized deployment with Docker Compose. Load-balanced API servers. Separate analytics database.

Phase 3: Regional Logistics

500-5000 trucks, 50+ partners, multi-region. Kubernetes orchestration. Sharded PostgreSQL by tenant. CDN for static assets. Event-driven microservices.

Phase 4: AI-Powered Orchestration

5000+ trucks, predictive dispatch, autonomous matching. ML inference services. Edge computing for GPS processing. Global load balancing.

9.1 Horizontal Scaling Components

- **Load-Balanced API Servers:** NGINX reverse proxy with least-connections algorithm, health checks, and automatic failover
- **Redis Caching:** Multi-tier cache (L1 in-memory, L2 Redis cluster) for dashboard data, truck statuses, and route calculations
- **Queue Workers:** Background job processing for notifications, analytics aggregation, and report generation via Bull/Redis queues
- **Event-Driven Architecture:** Decoupled services communicating via Kafka topics, enabling independent scaling of read and write paths
- **CDN for Assets:** Cloudflare or AWS CloudFront for static assets, reducing origin server load by 70-80%

10. Operational Pain Points Addressed

The platform directly solves twelve critical pain points endemic to fragmented logistics operations:

- **No Centralized Truck Visibility:** Real-time dashboard showing all partner trucks, statuses, and locations in unified view
- **Manual Dispatch Coordination:** Automated dispatch engine with intelligent truck-driver matching based on proximity, capacity, and availability
- **Viber/Messenger Dependency:** In-platform messaging, push notifications, and structured status updates replacing informal chat coordination
- **Dispatch Delays:** Sub-5-minute dispatch assignment vs. 30-120 minute manual coordination cycles
- **Idle Truck Inefficiency:** Utilization analytics identifying idle trucks and recommending return-load opportunities
- **Poor Fleet Utilization:** Capacity tracking and demand forecasting to optimize truck deployment
- **Double Booking of Trucks:** Real-time availability locks preventing concurrent assignments with optimistic concurrency control
- **No Realtime Operational Intelligence:** Live dashboards with exception alerting, delay predictions, and automated escalation
- **No Analytics Dashboards:** Comprehensive analytics covering utilization, delays, partner performance, fuel efficiency, and turnaround time
- **No Predictive Forecasting:** ML-based demand forecasting and predictive maintenance scheduling
- **Poor Maintenance Visibility:** Maintenance tracking with automated scheduling, compliance alerts, and downtime minimization
- **Lack of Partner Coordination:** Partner portal with fleet management, performance visibility, and collaborative dispatch workflows

11. Future Expansion Features

The platform roadmap includes advanced capabilities to maintain competitive advantage:

- **AI Dispatch Recommendations:** Machine learning models suggesting optimal truck-driver-route combinations based on historical performance, traffic patterns, and cargo characteristics
- **Predictive Maintenance:** IoT sensor integration and predictive algorithms forecasting component failures before they occur, reducing unplanned downtime by 40%
- **Return-Load Matching:** AI-powered identification of backhaul opportunities, reducing empty miles and increasing partner revenue
- **Dynamic Pricing:** Market-driven pricing engine adjusting rates based on demand, distance, cargo type, and partner availability
- **Smart Routing:** Real-time route optimization considering traffic, weather, road conditions, and delivery time windows
- **Driver Behavior Analytics:** Telematics-based scoring of driving patterns (speed, braking, idling) for safety and fuel efficiency improvement
- **Fuel Optimization:** Route and load optimization algorithms reducing fuel consumption by 15-25% through smarter dispatch decisions
- **IoT Truck Sensors:** Integration with temperature sensors, door sensors, weight sensors, and engine diagnostics for cargo integrity monitoring
- **Automated Invoicing:** Blockchain-verified delivery confirmation triggering automatic invoice generation and payment reconciliation
- **ERP Integrations:** Pre-built connectors for SAP, Oracle NetSuite, Microsoft Dynamics, and custom ERP systems via standardized API adapters
- **Port & Ferry Integrations:** Direct integration with port authority systems and ferry booking platforms for inter-island logistics coordination

12. Final System Vision

The long-term vision is to become the operational backbone of regional logistics networks across Southeast Asia. The platform evolves through four transformational stages:

- **Stage 1: Truck Visibility Dashboard** — Foundation layer providing real-time fleet visibility for single manufacturing companies
- **Stage 2: Dispatch Coordination Platform** — Multi-partner dispatch orchestration with automated matching and status tracking
- **Stage 3: Logistics Intelligence Platform** — AI-powered optimization, predictive analytics, and autonomous decision support
- **Stage 4: Regional Logistics Network** — Interconnected logistics ecosystem connecting manufacturers, distributors, warehouses, and transport partners across regions

Supported Verticals:

- Manufacturing logistics (raw materials, finished goods)
- Inter-island cargo (Philippine archipelago coordination)
- Construction logistics (heavy equipment, materials)
- Feed and flour distribution (agricultural supply chain)
- Warehouse coordination (inbound/outbound synchronization)
- Shared trucking ecosystems (capacity marketplace)

Document Conclusion: This architecture blueprint provides the technical foundation for building a world-class logistics coordination platform. By combining real-time visibility, intelligent dispatch, enterprise security, and scalable infrastructure, the platform transforms fragmented logistics operations into a unified, data-driven command center.